C# and .NET Fundamentals

This first roadmap topic covers the foundation needed before learning variables, operators, methods, OOP, collections, LINQ, async programming, ASP.NET Core, dependency injection, or architecture.

Index / Navigation

- Topic Coverage
- What Is .NET?
  - Definition
- Why Does .NET Exist?
- What Can We Build with .NET?
- .NET Is Cross-Platform
- What Is C#?
  - Definition
- C# Is Strongly Typed
- C# and .NET Are Not the Same Thing
- Current Learning Baseline
- .NET Framework vs .NET Core vs Modern .NET
  - .NET Framework
- .NET Core
- Modern .NET
- Comparison
- What Is the .NET SDK?
  - Definition
- SDK Example
- SDK vs Runtime
  - .NET SDK
  - .NET Runtime
- SDK vs Runtime Flow
- Simple Rule
- What Is the CLR?
  - Definition
- What Does the CLR Do?
- CLR Execution Concept
- What Is Managed Code?
  - Definition
- Why Is It Called Managed?
- What Is Unmanaged Code?
  - Definition
- Managed vs Unmanaged
- What Is CIL / IL?
  - Definition
- Why Have Intermediate Language?
- CIL Example Concept
- Metadata
- What Is JIT?
  - Definition
- Why Does JIT Exist?
- JIT Happens on Demand
- Example

Modern JIT Is More Advanced Than "Compile Once"
Complete C# Compilation Process
    Development Stage
    Runtime Stage
Compile Time vs Runtime
    Compile Time
Runtime
What Is an Assembly?
    Definition
Assembly Example
What Is Inside an Assembly?
Assembly Manifest
Why Assemblies Matter
DLL vs EXE
    DLL
EXE
What Is an Entry Point?
ASP.NET Core Preview
What Is a Namespace?
    Definition
Why Do We Need Namespaces?
Namespace Is Not the Same as Assembly
Folder vs Namespace
What Is a Project?
    Definition
What Is a .csproj File?
Understanding the Project File
    Project SDK
TargetFramework
Nullable
ImplicitUsings
PackageReference
ProjectReference
What Is a Solution?
    Definition
Solution vs Project
Why Use Multiple Projects?
What Is NuGet?
    Definition
NuGet Flow
Adding a Package
Package Restore
Important Package Rule
What Is the .NET CLI?
    Definition
Important CLI Commands

- Creating a Console Project
- What Happens During dotnet build?
- What Happens During dotnet run?
- What Is MSBuild?
- Debug Build
- Release Build
- Debug Does Not Mean "Logging Enabled"
- What Are PDB Files?
- IDE vs .NET
- Complete Application Development Flow
- Full Fundraising Example
- Source Code, Project, Assembly and Process
  - Source Code
  - Project
  - Assembly
  - Process
- Namespace vs Project vs Assembly
- Runtime Loading
- Type Identity and Assemblies
- JIT vs AOT
- Why .NET Can Optimize Code at Runtime
- Type Safety
- Base Class Library
- Framework vs Library vs Runtime
  - Runtime
  - Library
  - Framework
  - SDK
- Common Beginner Mistakes
  - Thinking C# and .NET Are Identical
  - Thinking the Compiler Directly Produces Only CPU Code
  - Thinking CLR Is the C# Compiler
  - Thinking SDK and Runtime Are the Same
  - Thinking Namespace Means Folder
  - Thinking Namespace Means Assembly
  - Thinking Every Project Produces an EXE
  - Thinking More Projects Automatically Mean Better Architecture
- Best Practices
- Fundraising Platform Connection
- Real Development Example
- Beginner-to-Advanced View of the Same Application
  - Beginner View
  - Intermediate View
  - Advanced View
- When This Knowledge Becomes Useful
- Interview Questions
  - What is .NET?

- What is the difference between C# and .NET?
- What is the difference between .NET Framework, .NET Core, and modern .NET?
- What is CLR?
- What is managed code?
- What is unmanaged code?
- What is CIL or IL?
- What is JIT compilation?
- Explain the C# execution process.
- What is an assembly?
- What is the difference between namespace and assembly?
- What is the .NET SDK?
- What is the difference between SDK and Runtime?
- What is a .csproj file?
- What is NuGet?
- What is the difference between a solution and a project?
- What is the difference between Debug and Release?

Practical Exercises

Quick Revision

Complete Final Flow

Topic Coverage

This first roadmap topic covers the foundation needed before learning variables, operators, methods, OOP, collections, LINQ, async programming, ASP.NET Core, dependency injection, or architecture.

EXAMPLE / FLOW

```
.NET
    ->
C#
    ->
.NET SDK
    ->
Compiler
    ->
CIL / IL
    ->
Assembly
    ->
CLR
    ->
JIT
    ->
Native Machine Code
    ->
Application Runs
```

We will understand:

- what .NET actually is;
- what C# is;
- .NET Framework vs .NET Core vs modern .NET;
- SDK vs Runtime;
- CLR;
- managed and unmanaged code;
- CIL / IL;
- JIT compilation;
- complete C# compilation and execution flow;
- assemblies;
- `.dll` and `.exe`;
- namespaces;
- projects and `.csproj`;
- solutions;
- NuGet;
- .NET CLI;

- Debug and Release builds;
- how a real .NET application starts and executes.

For current learning, we will use .NET 10 LTS and C# 14. .NET 10 is an LTS release supported for three years, and C# 14 is the current language version associated with .NET 10.

What Is .NET?

Definition

.NET is Microsoft's development platform for building and running applications.

It gives developers:

- a runtime to execute applications;
- libraries containing reusable functionality;
- compilers and build tools;
- development tools;
- package management;
- application frameworks such as ASP.NET Core.

C# is a programming language.

.NET is the platform on which your C# application normally builds and runs.

That difference is important.

EXAMPLE / FLOW

```
C#
=
Programming Language

.NET
=
Platform + Runtime + Libraries + Development Tools
```

For example, when creating the Fundraising Web API, you write code in C#:

C#

```
public class GiftService
{
    public decimal CalculateRemainingAmount(
        decimal giftAmount,
        decimal allocatedAmount)
    {
        return giftAmount - allocatedAmount;
    }
}
```

C# defines things such as:

EXAMPLE / FLOW

```
class
public
decimal
return
method
```

But functionality such as:

C#

```
Console.WriteLine();
DateTime.UtcNow;
Guid.NewGuid();
List<string>();
HttpClient;
```

comes from libraries available through .NET.

Why Does .NET Exist?

Imagine building an application without a development platform.

You would have to solve many low-level problems yourself:

EXAMPLE / FLOW

```
Memory management
Thread management
File access
Networking
HTTP communication
JSON processing
Exception handling infrastructure
Operating-system interaction
Type loading
Code execution
Security
```

.NET provides a standardized environment for solving these problems.

Your application developer can therefore concentrate mainly on:

EXAMPLE / FLOW

```
Fundraising business logic
        ->
Donors
        ->
Campaigns
        ->
Gifts
        ->
Gift allocations
        ->
Payments
```

instead of manually implementing low-level operating-system functionality.

What Can We Build with .NET?

.NET can be used for many kinds of applications.

Examples include:

EXAMPLE / FLOW

```
Console Applications

ASP.NET Core Web APIs

Web Applications

Worker Services

Background Services

Cloud Applications

Desktop Applications

Mobile Applications

Microservices

gRPC Services

Real-time Applications

Libraries

Testing Projects
```

For your learning path, our main focus will eventually become:

EXAMPLE / FLOW

```
C#
    ->
.NET
    ->
ASP.NET Core
    ->
Web API
    ->
Application Services
    ->
Database
```

.NET Is Cross-Platform

Modern .NET applications can run on supported versions of:

EXAMPLE / FLOW

```
Windows
Linux
macOS
```

This matters because a developer might develop an application on Windows but deploy it to a Linux container or cloud server.

Conceptually:

EXAMPLE / FLOW

```
C# Application
      ->
.NET
      ->
Windows

or

C# Application
      ->
.NET
      ->
Linux

or

C# Application
      ->
.NET
      ->
macOS
```

The managed execution model is one reason the same managed application code can target different supported machine architectures: the source is compiled to intermediate language, and the runtime ultimately produces machine-specific code.

What Is C#?

Definition

C# is a modern, strongly typed, object-oriented programming language used heavily with .NET.

C# gives us the language syntax used to describe our application.

For example:

C#

```
public sealed class Gift
{
    public long GiftId { get; init; }

    public decimal TotalAmount { get; private set; }

    public void ChangeAmount(decimal newAmount)
    {
        if (newAmount <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(newAmount));
        }

        TotalAmount = newAmount;
    }
}
```

C# provides the language features used above:

EXAMPLE / FLOW

```
class
properties
methods
if
exceptions
access modifiers
types
```

C# Is Strongly Typed

C# normally knows the type of a value.

Example:

C#

```
decimal giftAmount = 500.00m;
```

The variable:

EXAMPLE / FLOW

```
giftAmount
```

has type:

EXAMPLE / FLOW

```
decimal
```

You cannot simply assign unrelated text:

C#

```
giftAmount = "John";
```

That does not satisfy the type rules.

Strong typing helps detect many mistakes during compilation instead of waiting until the application is running.

Later we will study this deeply under:

EXAMPLE / FLOW

```
Variables and Data Types
Value Types and Reference Types
Generics
Nullable Types
```

C# and .NET Are Not the Same Thing

This is a common beginner interview question.

Consider:

EXAMPLE / FLOW

```
C#
    ↓
Language used to write application code

.NET
    ↓
Platform used to build and execute application code
```

A simple analogy is:

EXAMPLE / FLOW

```
C#       -> Language
.NET     -> Environment/platform
```

Do not say:

REMEMBER C# is .NET.

That is incorrect.

A better explanation is:

REMEMBER C# is one of the main programming languages used to build applications on the .NET platform.

Current Learning Baseline

For these notes we will use:

EXAMPLE / FLOW

```
.NET 10 LTS
C# 14
ASP.NET Core 10 later in roadmap
```

Microsoft documents .NET 10 as an LTS release and C# 14 as the current C# release supported on .NET 10.

You do not need to memorize every new C# 14 feature now.

Features such as:

EXAMPLE / FLOW

```
Extension members
field-backed properties
null-conditional assignment
additional partial members
Span-related improvements
```

belong later in the roadmap under advanced C#.

For now, understand:

EXAMPLE / FLOW

```
.NET version
and
C# language version
```

are related but are not the same concept.

.NET Framework vs .NET Core vs Modern .NET

This is another important interview topic.

.NET Framework

`.NET Framework` is the older Windows-focused .NET implementation.

It was widely used for:

EXAMPLE / FLOW

```
ASP.NET applications
Windows applications
enterprise applications
WCF
Web Forms
older MVC applications
```

It still exists for maintaining existing applications.

But new cross-platform backend development normally uses modern `.NET`.

.NET Core

Microsoft later introduced `.NET Core`.

Its major goals included:

EXAMPLE / FLOW

```
Cross-platform development
Open-source development
Higher performance
Cloud-friendly applications
Modular deployment
Modern web development
```

Applications could run on:

EXAMPLE / FLOW

```
Windows
Linux
macOS
```

ASP.NET Core became the modern web framework.

Modern .NET

Starting with `.NET 5`, Microsoft moved away from continuing the `.NET Core` naming.

Modern versions are named:

EXAMPLE / FLOW

```
.NET 5
.NET 6
.NET 7
.NET 8
.NET 9
.NET 10
...
```

So when we say:

EXAMPLE / FLOW

```
.NET 10
```

we mean modern unified .NET.

Do not call it:

EXAMPLE / FLOW

```
.NET Core 10
```

The correct name is:

EXAMPLE / FLOW

```
.NET 10
```

Comparison

Feature	.NET Framework	.NET Core	Modern .NET
Era	Older	Transition generation	Current
Windows	Yes	Yes	Yes
Linux	Generally no	Yes	Yes
macOS	Generally no	Yes	Yes
ASP.NET Core	Limited relationship	Yes	Yes
Open source	Partial	Yes	Yes
Cloud/container focus	Limited	Strong	Strong
New backend development	Usually no	Older versions	Yes

For your learning:

EXAMPLE / FLOW

```
Do not start with .NET Framework.

Learn modern .NET.
```

You may later encounter .NET Framework at work when maintaining legacy systems.

What Is the .NET SDK?

Definition

The .NET SDK — Software Development Kit — contains the tools required to develop .NET applications.

Microsoft describes the SDK as including the .NET CLI, runtime and libraries, and the `dotnet` driver.

The SDK allows you to:

EXAMPLE / FLOW

```
Create projects
Build applications
Run applications
Restore packages
Run tests
Publish applications
Create NuGet packages
```

SDK Example

After installing the SDK, you can execute:

EXAMPLE / FLOW

```
dotnet --version
```

and commands such as:

EXAMPLE / FLOW

```
dotnet new console
dotnet build
dotnet run
```

Without the development SDK, you would not have the complete normal development toolset.

SDK vs Runtime

This distinction is extremely important.

.NET SDK

Used mainly when:

EXAMPLE / FLOW

```
Developing
Compiling
Building
Testing
Publishing
```

It contains development tooling.

.NET Runtime

Used mainly when:

EXAMPLE / FLOW

```
Running an already built application
```

SDK vs Runtime Flow

EXAMPLE / FLOW

```
Developer Machine

Source Code
    ->
.NET SDK
    ->
Build
    ->
Application
```

Then:

EXAMPLE / FLOW

```
Server

Built Application
    ->
.NET Runtime
    ->
Application Runs
```

The SDK itself includes runtime components needed during development.

Simple Rule

If you are a developer:

EXAMPLE / FLOW

```
Install SDK
```

If a machine only needs to run a framework-dependent application:

EXAMPLE / FLOW

```
Runtime may be enough
```

Deployment models can change this—for example, self-contained deployment can package runtime components with the application—but that belongs later under deployment.

What Is the CLR?

Definition

SECTION CLR means Common Language Runtime.

It is the managed execution environment responsible for running .NET managed code.

You can think of it as the engine underneath a running managed .NET application.

EXAMPLE / FLOW

```
Your C# Application
        ->
CLR
        ->
Operating System / Hardware
```

The runtime provides important services while application code executes. Microsoft documents managed execution as including services such as garbage collection, security/runtime verification concepts, interoperability, debugging support, deployment/versioning support, and JIT compilation.

What Does the CLR Do?

Important responsibilities include:

EXAMPLE / FLOW

```
Execute managed code

JIT compile intermediate code

Manage memory

Run garbage collection

Handle exceptions

Enforce type-safety-related runtime behavior

Manage threads/runtime execution

Load types and assemblies

Support interoperability with native code
```

Do not memorize this as an empty list.

Understand why each exists.

For example:

EXAMPLE / FLOW

```
new Gift()
```

creates an object.

You do not manually allocate and later free the object's normal managed memory as you might in lower-level manual-memory models.

The CLR and garbage collector participate in managing that object's lifetime.

We will study the garbage collector deeply in the later CLR, Garbage Collection and Memory Management topic.

CLR Execution Concept

Suppose the application contains:

C#

```
public static decimal CalculateNetGift(
    decimal giftAmount,
    decimal fee)
{
    return giftAmount - fee;
}
```

The high-level process is:

EXAMPLE / FLOW

```
C# Source
    ->
Compiler
    ->
Intermediate Language
    ->
CLR loads application
    ->
JIT compiles method
    ->
CPU executes native instructions
```

What Is Managed Code?

Definition

SECTION Managed code is code whose execution is managed by the .NET runtime.

Your normal C# application code is managed code.

Example:

C#

```
var gift = new Gift();
```

The runtime provides services around execution such as:

EXAMPLE / FLOW

```
Memory management
Garbage collection
Runtime type information
Exception infrastructure
JIT compilation
```

Microsoft's managed execution documentation describes source being compiled to CIL and then executed under CLR runtime services.

Why Is It Called Managed?

Because application execution happens under runtime management.

Conceptually:

EXAMPLE / FLOW

```
Your Code
    ->
CLR manages execution environment
    ->
Machine Code executes
```

It does not mean:

REMEMBER The runtime controls every operation of your application.

It means the code participates in the managed runtime environment.

What Is Unmanaged Code?

Definition

Unmanaged/native code executes outside the normal managed CLR environment.

Examples may include:

EXAMPLE / FLOW

```
Native operating-system APIs
C/C++ native libraries
Certain third-party native libraries
Low-level platform components
```

A C# application can sometimes communicate with unmanaged code using interoperability mechanisms.

Example technologies include:

EXAMPLE / FLOW

```
P/Invoke
COM interoperability
Native library interop
```

You do not need these for normal beginner ASP.NET Core development.

Managed vs Unmanaged

EXAMPLE / FLOW

```
Managed C# Code
       ↓
CLR services
       ↓
Native execution
```

Compared with:

EXAMPLE / FLOW

```
Native / Unmanaged Code
        ↓
Operating system / native environment
```

You will primarily write managed code as a .NET web developer.

What Is CIL / IL?

Definition

When the C# compiler builds normal managed C# code, it does not normally translate every method directly into final machine instructions for your current CPU.

Instead, it produces an intermediate instruction representation called:

EXAMPLE / FLOW

```
CIL
```

which means:

EXAMPLE / FLOW

```
Common Intermediate Language
```

People also commonly say:

EXAMPLE / FLOW

```
IL
Intermediate Language
```

Microsoft describes CIL as a CPU-independent instruction set that can later be efficiently converted to native code.

Why Have Intermediate Language?

Imagine compiling directly for one processor.

You might create:

EXAMPLE / FLOW

```
Windows x64 executable
```

and need a different compilation for:

EXAMPLE / FLOW

```
Linux ARM64
```

Managed .NET's intermediate representation provides another layer:

EXAMPLE / FLOW

```
C# Source
      ->
CIL
      ->
Runtime on target environment
      ->
Native Machine Code
```

The runtime understands the target environment.

CIL Example Concept

You write:

C#

```
int total = 10 + 20;
```

You understand C#.

The processor understands machine instructions.

The .NET build/execution pipeline introduces intermediate instructions between those levels.

EXAMPLE / FLOW

```
C#
    ->
CIL
    ->
Native Instructions
```

You normally do not manually write CIL when developing ASP.NET Core applications.

But understanding it explains how .NET executes code.

Metadata

The compiler also generates metadata.

Metadata describes things such as:

EXAMPLE / FLOW

```
Types
Classes
Methods
Properties
Method signatures
Referenced types
Assembly information
```

Microsoft documents the generated CIL and metadata as being stored together in executable/assembly files used by the runtime.

This metadata makes features such as runtime type inspection possible.

Later, when you study reflection:

C#

```
typeof(Gift).GetProperties();
```

reflection can inspect metadata describing the `Gift` type.

What Is JIT?

Definition

SECTION JIT means Just-In-Time compiler.

Its job is to convert managed intermediate code into native machine code the processor can execute.

EXAMPLE / FLOW

```
CIL
    ->
JIT
    ->
Native Machine Code
```

Why Does JIT Exist?

Your CPU cannot directly execute normal CIL instructions.

It needs machine code for the target processor architecture.

So before managed method code executes, native instructions are needed.

Microsoft explains that the JIT compiler converts CIL into processor-specific native code at runtime.

JIT Happens on Demand

A useful beginner model is:

EXAMPLE / FLOW

```
Application starts
    ->
Method needed
    ->
JIT compiles method
    ->
Native code stored for process
    ->
Method executes
```

Microsoft documents that methods containing CIL are normally JIT-compiled when first called, with the generated native code reused for subsequent calls in that process.

Example

Application:

C#

```
static void Main()
{
    CreateGift();
    CreateGift();
}

static void CreateGift()
{
    Console.WriteLine("Gift created");
}
```

Simplified runtime concept:

EXAMPLE / FLOW

```
First CreateGift() call
    ->
JIT compiles CreateGift
    ->
Native code runs

Second CreateGift() call
    ->
Already compiled native code
    ->
Run existing native code
```

This is a conceptual model; modern .NET also contains more sophisticated runtime optimization behavior that we will study later.

Modern JIT Is More Advanced Than "Compile Once"

For beginner learning:

EXAMPLE / FLOW

```
IL -> JIT -> Native code
```

is enough.

At advanced level, modern .NET can perform runtime optimizations such as tiered compilation and increasingly optimized versions of frequently executed methods.

.NET 10 also includes JIT improvements such as enhanced inlining, devirtualization, stack-allocation optimizations, and other code-generation improvements.

You do not need to optimize code around these features manually yet.

Complete C# Compilation Process

This is one of the most important flows in this topic.

Suppose we create:

C#

```
public class GiftCalculator
{
    public decimal Calculate(
        decimal totalAmount,
        decimal refundAmount)
    {
        return totalAmount - refundAmount;
    }
}
```

Development Stage

EXAMPLE / FLOW

```
GiftCalculator.cs
        ->
C# Compiler
        ->
CIL + Metadata
        ->
Assembly
```

Runtime Stage

EXAMPLE / FLOW

```
Assembly
    ->
CLR loads assembly
    ->
Method needed
    ->
JIT compiler
    ->
Native machine instructions
    ->
CPU executes instructions
```

Full flow:

EXAMPLE / FLOW

```
C# Source Code
    ->
C# Compiler
    ->
CIL + Metadata
    ->
.dll / .exe Assembly
    ->
CLR
    ->
JIT
    ->
Native Machine Code
    ->
CPU
    ->
Application Result
```

This managed execution sequence is the foundation described by Microsoft's .NET managed execution documentation.

Compile Time vs Runtime

Understanding this distinction will help throughout the entire roadmap.

Compile Time

Occurs when code is being built.

Example error:

C#

```
decimal amount = "ABC";
```

The compiler can detect incompatible types.

EXAMPLE / FLOW

```
Source Code
    ->
Compiler
    ->
Compile-Time Error
```

The application cannot successfully build until the problem is corrected.

Runtime

Runtime means the application has started executing.

Example:

C#

```
int number = 0;
int result = 100 / number;
```

This can compile.

But while executing:

EXAMPLE / FLOW

```
100 / 0
```

causes an exception.

EXAMPLE / FLOW

```
Compilation
    ->
Successful
    ->
Application starts
    ->
Problem occurs
    ->
Runtime exception
```

This difference becomes extremely important when we study:

EXAMPLE / FLOW

```
Type system
Exceptions
Generics
dynamic
Reflection
ASP.NET Core
```

What Is an Assembly?

Definition

An assembly is a fundamental compiled unit of a .NET application or library.

Assemblies commonly appear as:

EXAMPLE / FLOW

```
.dll

or

.exe
```

Microsoft describes assemblies as fundamental units of deployment, versioning, reuse, and runtime type information.

Assembly Example

Suppose your solution eventually contains:

EXAMPLE / FLOW

```
Fundraising.Api

Fundraising.Application

Fundraising.Domain

Fundraising.Infrastructure
```

After building, you might see:

EXAMPLE / FLOW

```
Fundraising.Api.dll

Fundraising.Application.dll

Fundraising.Domain.dll

Fundraising.Infrastructure.dll
```

Each project commonly produces its own assembly.

What Is Inside an Assembly?

Conceptually:

EXAMPLE / FLOW

```
Assembly
    ->
CIL
    +
Metadata
    +
Manifest
    +
Resources
```

The assembly manifest contains information that helps identify the assembly and its contents/references.

Microsoft notes that assemblies contain types/resources that form a logical unit and that the runtime uses assembly metadata to understand implementations and references.

Assembly Manifest

The assembly manifest is metadata describing the assembly itself.

Conceptually it can identify:

EXAMPLE / FLOW

```
Assembly identity
Version information
Files
Referenced assemblies
Types/resources information
```

Think:

EXAMPLE / FLOW

```
Assembly
    ->
Manifest tells runtime what this assembly is
```

Why Assemblies Matter

Assemblies allow an application to be divided into reusable components.

For example:

EXAMPLE / FLOW

```
Fundraising.Api
        ->
references
        ->
Fundraising.Application
        ->
references
        ->
Fundraising.Domain
```

This is very important when we later learn application architecture.

DLL vs EXE

DLL

DLL means:

EXAMPLE / FLOW

```
Dynamic Link Library
```

In .NET, library projects commonly produce `.dll` assemblies.

Example:

EXAMPLE / FLOW

```
Fundraising.Domain.dll
```

It normally contains reusable classes but does not independently represent the normal application entry point.

EXE

An executable application can have an executable entry point.

Historically and depending on publish/build settings, executable artifacts may appear as `.exe` or platform-specific executables, while managed application assemblies may still exist as `.dll` files.

For learning:

EXAMPLE / FLOW

```
Library
    ->
usually .dll

Runnable app
    ->
has application entry point
```

Do not assume that every modern .NET application must be represented internally only by a traditional Windows `.exe`.

What Is an Entry Point?

The entry point is where application execution begins.

Historically you commonly saw:

C#

```
public static void Main(string[] args)
{
    Console.WriteLine("Application started");
}
```

`Main` is the application's entry point.

Modern C# also supports top-level statements.

Example:

C#

```
Console.WriteLine("Application started");
```

The compiler generates the necessary entry-point structure for you.

ASP.NET Core Preview

Later your `Program.cs` may look similar to:

C#

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

var app = builder.Build();

app.MapControllers();

app.Run();
```

You do not need to understand this yet.

Later we will break down:

EXAMPLE / FLOW

```
builder
DI container
middleware
routing
app.Run()
```

in the ASP.NET Core topics.

For now, understand that `Program.cs` participates in starting the application.

What Is a Namespace?

Definition

A namespace organizes related types and helps prevent naming conflicts.

Example:

C#

```
namespace Fundraising.Domain.Gifts;

public class Gift
{
}
```

The full type name conceptually becomes:

EXAMPLE / FLOW

```
Fundraising.Domain.Gifts.Gift
```

Why Do We Need Namespaces?

Imagine two libraries contain a class named:

EXAMPLE / FLOW

```
Task
```

Without qualification, the names could be confusing.

Namespaces allow:

EXAMPLE / FLOW

```
System.Threading.Tasks.Task
```

and your own:

EXAMPLE / FLOW

```
Fundraising.Domain.Tasks.TaskItem
```

to be distinguished.

Namespace Is Not the Same as Assembly

Important distinction:

EXAMPLE / FLOW

```
Namespace
=
Logical organization of types

Assembly
=
Compiled deployment/runtime unit
```

One assembly can contain many namespaces.

A namespace can also conceptually be spread across types compiled from multiple files/projects depending on how code is organized.

Do not think:

EXAMPLE / FLOW

```
One namespace = one assembly
```

That is incorrect.

Folder vs Namespace

Developers often align folder structure and namespace:

EXAMPLE / FLOW

```
Gifts/
    Gift.cs
```

with:

C#

```
namespace Fundraising.Domain.Gifts;
```

That is a good organization convention.

But a namespace is not physically required to match the folder name.

What Is a Project?

Definition

A .NET project groups:

EXAMPLE / FLOW

```
Source files
References
Dependencies
Build configuration
Target framework
Package references
Compiler/build settings
```

A C# project normally has a:

EXAMPLE / FLOW

```
.csproj
```

file.

What Is a .csproj File?

`.csproj` means:

EXAMPLE / FLOW

```
C# Project File
```

Modern .NET uses SDK-style project files based on MSBuild XML. Microsoft documents `.csproj` as an XML-based project format that references the SDK and controls build behavior.

A simple file might look like:

XML

```xml
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

Understanding the Project File

Project SDK

XML

```xml
<Project Sdk="Microsoft.NET.Sdk">
```

This says the project uses the normal .NET SDK project system.

For an ASP.NET Core project, you commonly see:

```xml
<Project Sdk="Microsoft.NET.Sdk.Web">
```

Microsoft documents `Microsoft.NET.Sdk` and `Microsoft.NET.Sdk.Web` as SDK choices for normal .NET and web projects respectively.

TargetFramework

```xml
<TargetFramework>net10.0</TargetFramework>
```

This means the project targets:

```
.NET 10
```

The target framework affects:

```
Available APIs
Runtime compatibility
Compiler/runtime expectations
Package compatibility
```

Nullable

```xml
<Nullable>enable</Nullable>
```

enables nullable reference type analysis.

Later we will study:

```
string?
```

versus:

```
string
```

in detail.

ImplicitUsings

```xml
<ImplicitUsings>enable</ImplicitUsings>
```

allows the SDK to automatically provide commonly used `using` directives depending on the project type.

This reduces repetitive boilerplate.

PackageReference

When NuGet packages are used, the project can contain:

```xml
<ItemGroup>
    <PackageReference
        Include="Some.Package"
        Version="1.2.3" />
</ItemGroup>
```

This tells the build system that the project depends on that package.

ProjectReference

If:

```
Fundraising.Api
```

uses:

```
Fundraising.Application
```

there may be a project reference:

```xml
<ItemGroup>
    <ProjectReference
        Include="..\Fundraising.Application\Fundraising.Application.csproj" />
</ItemGroup>
```

Conceptually:

EXAMPLE / FLOW

```
Fundraising.Api
      ->
references
      ->
Fundraising.Application
```

What Is a Solution?

Definition

A solution organizes multiple related projects.

For your learning project:

EXAMPLE / FLOW

```
FundraisingPlatform
    |
    |-- Fundraising.Api
    |
    |-- Fundraising.Application
    |
    |-- Fundraising.Domain
    |
    `-- Fundraising.Infrastructure
```

Each child can be a separate project.

The solution gives development tools a way to work with them together.

Solution vs Project

Do not confuse them.

EXAMPLE / FLOW

```
Solution
    ->
contains / organizes projects

Project
    ->
contains source code and build configuration

Project
    ->
usually builds into assembly
```

Example:

EXAMPLE / FLOW

```
FundraisingPlatform.sln
        |
        |-- Fundraising.Api.csproj
        |-- Fundraising.Application.csproj
        |-- Fundraising.Domain.csproj
        `-- Fundraising.Infrastructure.csproj
```

Why Use Multiple Projects?

A tiny console program may need only one project.

A professional backend may use separate projects to enforce responsibilities.

For example:

EXAMPLE / FLOW

```
Fundraising.Domain
    ->
Business concepts

Fundraising.Application
    ->
Use cases

Fundraising.Infrastructure
    ->
Database/external services

Fundraising.Api
    ->
HTTP/API boundary
```

Do not create many projects just because more projects look professional.

Use boundaries when they provide real organizational or architectural value.

What Is NuGet?

Definition

SECTION NuGet is the package-management system commonly used by .NET.

Instead of manually copying third-party DLL files into every project, you can reference packages.

Examples of ecosystem packages may provide:

EXAMPLE / FLOW

```
Testing libraries
Database providers
Logging integrations
Validation libraries
HTTP resiliency functionality
```

NuGet Flow

EXAMPLE / FLOW

```
Your Project
    ->
PackageReference
    ->
NuGet Restore
    ->
Package downloaded/resolved
    ->
Assembly/reference available
    ->
Project builds
```

Adding a Package

CLI form:

EXAMPLE / FLOW

```
dotnet add package PackageName
```

Modern tooling can also manage packages through Visual Studio.

The project file then records the package reference.

Package Restore

When opening/cloning a project, package files may not all be committed into source control.

Instead:

EXAMPLE / FLOW

```
.csproj
    ->
contains package references
    ->
dotnet restore
    ->
dependencies restored
```

`dotnet build` can also perform restore when necessary.

Important Package Rule

Do not add a NuGet package for functionality already available cleanly in the platform simply because a package exists.

Every external dependency can create:

EXAMPLE / FLOW

```
Maintenance responsibility
Version dependency
Security responsibility
Compatibility concerns
Upgrade work
```

Use packages intentionally.

What Is the .NET CLI?

Definition

The .NET Command-Line Interface allows you to create, build, run, test, restore, and publish .NET applications from a terminal.

It is included with the .NET SDK.

The main command begins with:

EXAMPLE / FLOW

```
dotnet
```

Important CLI Commands

Check installed SDK:

EXAMPLE / FLOW

```
dotnet --version
```

List SDKs:

EXAMPLE / FLOW

```
dotnet --list-sdks
```

Create application:

EXAMPLE / FLOW

```
dotnet new console
```

Build:

EXAMPLE / FLOW

```
dotnet build
```

Run:

EXAMPLE / FLOW

```
dotnet run
```

Restore dependencies:

EXAMPLE / FLOW

```
dotnet restore
```

Run tests:

EXAMPLE / FLOW

```
dotnet test
```

Publish:

EXAMPLE / FLOW

```
dotnet publish
```

Creating a Console Project

Example:

EXAMPLE / FLOW

```
dotnet new console -n Fundraising.Learning
```

Conceptually:

EXAMPLE / FLOW

```
dotnet new
    ->
Template selected
    ->
Project files created
    ->
.csproj created
    ->
Program.cs created
```

Then:

EXAMPLE / FLOW

```
cd Fundraising.Learning
dotnet run
```

What Happens During dotnet build?

Simplified:

EXAMPLE / FLOW

```
Read .csproj
    ->
Resolve SDK
    ->
Restore/resolve dependencies
    ->
Compile C# files
    ->
Generate CIL + metadata
    ->
Create assembly
    ->
Write build output
```

A typical output folder might contain something resembling:

EXAMPLE / FLOW

```
bin/
    Debug/
        net10.0/
```

What Happens During dotnet run?

Conceptually:

EXAMPLE / FLOW

```
dotnet run
    ->
build project if needed
    ->
start application
    ->
runtime loads assembly
    ->
entry point executes
```

What Is MSBuild?

Underneath the project build process, modern `.NET` project files use MSBuild.

Your `.csproj` contains build configuration.

The selected SDK contributes MSBuild targets and tasks used to:

EXAMPLE / FLOW

```
Compile
Resolve references
Generate outputs
Pack
Publish
```

Microsoft describes project SDKs as collections of MSBuild targets/tasks responsible for building, packing, and publishing code.

You do not need to learn advanced MSBuild syntax as a beginner.

But you should know:

EXAMPLE / FLOW

```
.csproj
    ->
MSBuild + .NET SDK
    ->
build process
```

Debug Build

When developing locally, you commonly use:

EXAMPLE / FLOW

```
Debug
```

configuration.

Example:

EXAMPLE / FLOW

```
dotnet build --configuration Debug
```

Debug builds are designed primarily to support development and debugging.

You commonly get:

EXAMPLE / FLOW

```
Debug symbols
Easier debugging experience
Less aggressive optimization behavior than Release
```

Release Build

For production-oriented builds:

EXAMPLE / FLOW

```
dotnet build --configuration Release
```

Release configuration allows compiler/runtime build settings intended for optimized deployment scenarios.

Conceptually:

EXAMPLE / FLOW

```
Debug
    ->
development + debugging

Release
    ->
production-oriented build
```

Debug Does Not Mean "Logging Enabled"

Do not confuse:

EXAMPLE / FLOW

```
Debug build configuration
```

with:

EXAMPLE / FLOW

```
application log level
```

They are different concepts.

Logging is configured separately.

What Are PDB Files?

During development builds you may see:

EXAMPLE / FLOW

```
.pdb
```

files.

PDBs contain debugging symbol information that helps tools map executing code back to source locations.

This allows debugging features such as:

EXAMPLE / FLOW

```
Breakpoints
Call stacks
Source line mapping
```

You do not normally edit PDB files manually.

IDE vs .NET

Visual Studio is not .NET.

This distinction matters.

EXAMPLE / FLOW

```
Visual Studio
=
Development environment / IDE

.NET
=
Development platform/runtime

C#
=
Programming language
```

You can write/build .NET applications without Visual Studio by using:

EXAMPLE / FLOW

```
Editor
+
.NET SDK
+
dotnet CLI
```

Visual Studio simply provides a very productive integrated environment.

Complete Application Development Flow

Let's connect everything.

You write:

C#

```
namespace Fundraising.Learning;

public static class GiftCalculator
{
    public static decimal CalculateUnallocated(
        decimal giftAmount,
        decimal allocatedAmount)
    {
        return giftAmount - allocatedAmount;
    }
}
```

Development:

EXAMPLE / FLOW

```
GiftCalculator.cs
    ->
.csproj
    ->
.NET SDK
    ->
C# compiler
```

Compilation:

EXAMPLE / FLOW

```
C# source
    ->
CIL
    +
metadata
    ->
assembly
```

Execution:

EXAMPLE / FLOW

```
Assembly
    ->
CLR
    ->
JIT
    ->
native machine code
    ->
CPU
```

Result:

EXAMPLE / FLOW

```
Fundraising business operation executes
```

Full Fundraising Example

Consider this tiny console application:

C#

```
namespace Fundraising.Learning;

internal static class Program
{
    private static void Main()
    {
        const decimal giftAmount = 1_000m;
        const decimal allocatedAmount = 700m;

        decimal remainingAmount =
            CalculateRemainingAmount(
                giftAmount,
                allocatedAmount);

        Console.WriteLine(
            $"Remaining amount: {remainingAmount:C}");
    }

    private static decimal CalculateRemainingAmount(
        decimal giftAmount,
        decimal allocatedAmount)
    {
        return giftAmount - allocatedAmount;
    }
}
```

Business meaning:

EXAMPLE / FLOW

```
Gift
$1,000
    ->
Allocated
$700
    ->
Remaining
$300
```

Technical flow:

EXAMPLE / FLOW

```
Program.cs
    ->
C# compiler
    ->
CIL + metadata
    ->
Fundraising.Learning.dll
    ->
CLR
    ->
JIT
    ->
machine instructions
    ->
$300 result printed
```

This one small example already uses several things we will later study separately:

EXAMPLE / FLOW

```
namespace
class
method
decimal
const
parameter
return value
string interpolation
Console
```

Do not worry about mastering all of those now.

Source Code, Project, Assembly and Process

These four concepts are often confused.

Source Code

Files you write:

EXAMPLE / FLOW

```
Gift.cs
Program.cs
Campaign.cs
```

Project

Build definition:

EXAMPLE / FLOW

```
Fundraising.Domain.csproj
```

It groups source/configuration/dependencies.

Assembly

Compiled output:

EXAMPLE / FLOW

```
Fundraising.Domain.dll
```

Process

When an executable application actually runs, the operating system runs it inside a process.

Conceptually:

EXAMPLE / FLOW

```
Source Files
    ->
Project
    ->
Build
    ->
Assembly
    ->
Runtime
    ->
Process executes application
```

Namespace vs Project vs Assembly

Concept	Purpose
Namespace	Organize type names
Project	Organize/build source and dependencies

Concept	Purpose
Assembly	Compiled logical/deployment unit
Solution	Organize multiple projects

Example:

EXAMPLE / FLOW

```
Solution
FundraisingPlatform

Project
Fundraising.Domain

Namespace
Fundraising.Domain.Gifts

Class
Gift

Assembly
Fundraising.Domain.dll
```

Runtime Loading

When a running application requires code from another assembly:

EXAMPLE / FLOW

```
Application
    ->
Assembly reference
    ->
Runtime resolves required assembly
    ->
Assembly loaded when required
    ->
Types become available
```

Microsoft notes that assemblies are loaded into memory when required and provide runtime information about their contained types.

Type Identity and Assemblies

An advanced point:

A type's identity is not just its short class name.

The assembly is part of the runtime's understanding of type identity.

For example, two assemblies could theoretically contain similarly named:

EXAMPLE / FLOW

```
Fundraising.Gift
```

types and still represent different runtime types.

Microsoft explicitly notes that type identity includes the containing assembly.

You do not need to manage this manually in normal projects, but it explains why assemblies matter beyond being "just DLL files."

JIT vs AOT

For the core beginner model:

EXAMPLE / FLOW

```
CIL
    ->
JIT at runtime
```

But modern .NET also supports Ahead-of-Time compilation scenarios such as Native AOT.

Conceptually:

JIT

EXAMPLE / FLOW

```
CIL
    ->
Application starts
    ->
JIT
    ->
Native code
```

AOT concept

EXAMPLE / FLOW

```
Build/publish
    ->
Native code generated earlier
    ->
Application starts with less JIT dependency
```

Native AOT has deployment/startup advantages in suitable scenarios, but also different compatibility and design trade-offs.

Do not start optimizing a normal ASP.NET Core learning application around Native AOT.

We will revisit deployment/runtime optimization much later.

Why .NET Can Optimize Code at Runtime

One benefit of runtime compilation is that the runtime understands the actual target environment.

JIT compilation can generate code for the machine executing the application.

Modern JITs can also optimize frequently executed code using runtime knowledge.

.NET 10 includes additional JIT optimization work such as improved inlining and method devirtualization.

For application developers the important lesson is:

EXAMPLE / FLOW

```
Write clear correct code first.

Measure performance.

Optimize only where evidence shows a problem.
```

Type Safety

C# and the CLR use a rich type system.

For example:

C#

```
decimal amount = 100m;
```

The compiler/runtime know this is:

EXAMPLE / FLOW

```
System.Decimal
```

This enables:

EXAMPLE / FLOW

```
Compile-time validation
Runtime type information
Safe method invocation
Generic type safety
Reflection
```

Later topics on value/reference types, generics, covariance, reflection, and runtime behavior all build on this foundation.

Base Class Library

.NET provides a very large standard library.

You will regularly use namespaces such as:

EXAMPLE / FLOW

```
System

System.Collections.Generic

System.Linq

System.Threading.Tasks

System.Net.Http

System.Text.Json
```

Examples:

C#

```
var giftIds = new List<long>();

Guid id = Guid.NewGuid();

DateTime now = DateTime.UtcNow;
```

You did not implement:

EXAMPLE / FLOW

```
List<T>
Guid
DateTime
```

yourself.

They come from .NET libraries.

Framework vs Library vs Runtime

These terms are related but different.

Runtime

Executes managed applications.

EXAMPLE / FLOW

```
CLR/runtime
```

Library

Reusable code you call.

EXAMPLE / FLOW

```
List<T>
HttpClient
JsonSerializer
```

Framework

Provides a larger programming model and structure.

Example:

EXAMPLE / FLOW

```
ASP.NET Core
```

for web applications.

SDK

Development tools used to build.

EXAMPLE / FLOW

```
dotnet
compiler/build tooling
templates
```

Common Beginner Mistakes

Thinking C# and .NET Are Identical

Incorrect:

EXAMPLE / FLOW

```
C# = .NET
```

Better:

EXAMPLE / FLOW

```
C# = language

.NET = platform
```

Thinking the Compiler Directly Produces Only CPU Code

The normal managed compilation flow involves intermediate language and metadata before runtime execution/JIT compilation.

Thinking CLR Is the C# Compiler

The compiler and runtime have different jobs.

EXAMPLE / FLOW

```
Compiler
    ->
C# -> CIL

CLR/JIT
    ->
CIL -> execution/native code
```

Thinking SDK and Runtime Are the Same

SDK:

EXAMPLE / FLOW

```
Develop + build + run
```

Runtime:

EXAMPLE / FLOW

```
Run
```

The SDK includes runtime components but provides substantially more developer tooling.

Thinking Namespace Means Folder

Folder organization and namespaces commonly align.

They are not the same technical feature.

Thinking Namespace Means Assembly

Again:

EXAMPLE / FLOW

```
Namespace
=
logical naming organization

Assembly
=
compiled unit
```

Thinking Every Project Produces an EXE

Class-library projects usually produce DLL assemblies.

Web and executable project output/deployment behavior depends on project and publishing model.

Thinking More Projects Automatically Mean Better Architecture

This:

EXAMPLE / FLOW

```
30 projects
```

does not automatically mean good architecture.

Good architecture means:

EXAMPLE / FLOW

```
clear responsibilities
appropriate boundaries
manageable dependencies
```

Best Practices

Use the current supported .NET generation for new learning unless you have a specific legacy requirement.

For this roadmap:

EXAMPLE / FLOW

```
.NET 10 LTS
C# 14
```

Keep project files simple until a real requirement demands custom MSBuild configuration.

Use namespaces that communicate application/domain organization.

Prefer clear project boundaries instead of creating projects for every tiny class.

Keep package dependencies intentional.

Understand what a package adds before installing it.

Build through both Visual Studio and the CLI occasionally so you understand that the IDE is not the runtime or build platform itself.

Learn to inspect `.csproj` instead of treating it as a mysterious generated file.

Understand:

EXAMPLE / FLOW

```
Source
-> compiler
-> CIL
-> assembly
-> CLR
-> JIT
-> CPU
```

before moving into deeper runtime subjects.

Fundraising Platform Connection

Eventually your application may have:

EXAMPLE / FLOW

```
FundraisingPlatform.sln

Fundraising.Api
Fundraising.Application
Fundraising.Domain
Fundraising.Infrastructure
```

Flow:

EXAMPLE / FLOW

```
Angular / Client
    ->
Fundraising.Api
    ->
Fundraising.Application
    ->
Fundraising.Domain

Infrastructure
    ->
Database / external integration implementations
```

From today's topic, understand only the technical packaging:

EXAMPLE / FLOW

```
FundraisingPlatform.sln
    ->
contains projects

Fundraising.Domain.csproj
    ->
builds project

Gift.cs
    ->
C# source

Fundraising.Domain.dll
    ->
compiled assembly

CLR
    ->
runs managed application code
```

Later architecture topics will explain why those layers exist.

Real Development Example

Create solution:

EXAMPLE / FLOW

```
dotnet new sln -n FundraisingPlatform
```

Create domain project:

EXAMPLE / FLOW

```
dotnet new classlib -n Fundraising.Domain
```

Create application project:

EXAMPLE / FLOW

```
dotnet new classlib -n Fundraising.Application
```

Add projects to solution using your installed CLI tooling.

Then conceptually:

EXAMPLE / FLOW

```
Solution
    ->
Domain project
    ->
Application project
```

A future solution can add:

EXAMPLE / FLOW

```
Fundraising.Api
Fundraising.Infrastructure
Tests
```

Do not build all architecture now merely for this topic.

The point is to see how the .NET tooling concepts connect.

Beginner-to-Advanced View of the Same Application

Beginner View

EXAMPLE / FLOW

```
I write C#.
I run the application.
```

Correct, but incomplete.

Intermediate View

EXAMPLE / FLOW

```
C# Source
    ->
Compiler
    ->
CIL
    ->
CLR/JIT
    ->
Native Code
```

Advanced View

EXAMPLE / FLOW

```
SDK-style Project
    ->
MSBuild + SDK targets
    ->
C# compiler
    ->
CIL + metadata
    ->
Assembly manifest
    ->
Runtime assembly loading
    ->
CLR services
    ->
JIT / runtime optimization
    ->
Native CPU execution
```

You do not need to actively manipulate every layer.

But understanding them makes later topics much easier.

When This Knowledge Becomes Useful

You will use these concepts when troubleshooting things such as:

EXAMPLE / FLOW

```
"SDK not found"

"This application requires .NET..."

"Target framework is incompatible"

"Package cannot be restored"

"Assembly could not be loaded"

"Project reference missing"

"Namespace/type cannot be found"

"Works in Debug but behaves differently in Release"

"Application builds but does not run on deployment server"
```

Without the foundation, these errors look unrelated.

After learning the platform model, you can identify which layer is failing.

Interview Questions

What is .NET?

.NET is a development platform that provides a runtime, libraries, SDK tooling, and frameworks for building and running applications.

For example, a C# ASP.NET Core Web API is written using C# but built and executed using the .NET platform.

What is the difference between C# and .NET?

C# is a programming language.

.NET is the platform that provides the runtime, libraries, tools, and application frameworks used to build and run the application.

EXAMPLE / FLOW

```
C# -> Language

.NET -> Platform
```

What is the difference between .NET Framework, .NET Core, and modern .NET

`.NET Framework` is the older primarily Windows-oriented implementation.

`.NET Core` introduced the modern cross-platform, open-source implementation.

From `.NET 5` onward, the product is normally called simply `.NET`, such as `.NET 8`, `.NET 9`, or `.NET 10`.

For new backend development we normally learn modern `.NET`.

What is CLR?

CLR means Common Language Runtime.

It provides the managed execution environment for .NET code, including runtime services such as JIT compilation, memory management/garbage collection, exception/runtime services, type loading, and interoperability support.

What is managed code?

Managed code executes under the .NET managed runtime environment.

Typical C# application code is managed code and benefits from runtime services such as garbage collection and JIT compilation.

What is unmanaged code?

Unmanaged code executes outside the normal managed CLR environment.

Native operating-system APIs or native C/C++ libraries are common examples.

.NET code can interact with unmanaged code through interop mechanisms when required.

What is CIL or IL?

CIL means Common Intermediate Language.

The C# compiler normally converts C# source into CPU-independent intermediate instructions plus metadata. The runtime later converts the needed intermediate code into native instructions for execution.

What is JIT compilation?

JIT means Just-In-Time compilation.

The JIT compiler converts CIL into native machine code when the application needs to execute that code.

The resulting native method code can then execute on the processor.

Explain the C# execution process.

A good interview explanation is:

EXAMPLE / FLOW

```
C# source code
    ->
C# compiler
    ->
CIL + metadata
    ->
Assembly
    ->
CLR loads assembly
    ->
JIT compiles required methods
    ->
Native machine code
    ->
CPU executes
```

What is an assembly?

An assembly is a compiled logical unit of a .NET application or library.

Assemblies commonly appear as `.dll` or executable artifacts and contain managed code, metadata, manifest information, and potentially resources.

What is the difference between namespace and assembly?

A namespace logically organizes type names.

An assembly is a compiled/deployment/runtime unit.

Many namespaces can exist inside one assembly.

What is the .NET SDK?

The .NET SDK is the development toolkit used to create, build, test, run, package, and publish .NET applications.

It includes tools such as the .NET CLI together with runtime/libraries needed for development.

What is the difference between SDK and Runtime?

SDK:

EXAMPLE / FLOW

```
used to develop and build applications
```

Runtime:

EXAMPLE / FLOW

```
used to execute applications
```

Developers normally install the SDK.

What is a .csproj file?

It is the C# project file.

It uses MSBuild XML and contains information such as:

EXAMPLE / FLOW

```
target framework
SDK
package references
project references
compiler/build configuration
```

Modern .NET projects use SDK-style project files.

What is NuGet?

NuGet is the package-management ecosystem used by .NET projects.

It lets projects declare and restore reusable libraries as package dependencies rather than manually copying dependency binaries.

What is the difference between a solution and a project?

A project contains source code and build configuration for one buildable unit.

A solution organizes multiple related projects.

Example:

EXAMPLE / FLOW

```
FundraisingPlatform.sln
    |
    |-- Fundraising.Api.csproj
    |-- Fundraising.Application.csproj
    |-- Fundraising.Domain.csproj
    `-- Fundraising.Infrastructure.csproj
```

What is the difference between Debug and Release?

Debug configuration is designed primarily for development and debugging.

Release configuration is intended for optimized production-oriented builds.

The exact generated/runtime behavior can depend on project and runtime configuration, so do not treat the distinction as simply "Debug is slow and Release is fast."

Practical Exercises

After studying this topic, you should be able to perform these tasks yourself.

Create a console application:

EXAMPLE / FLOW

```
dotnet new console -n Fundraising.Learning
```

Build:

EXAMPLE / FLOW

```
dotnet build
```

Run:

EXAMPLE / FLOW

```
dotnet run
```

Create a class library:

EXAMPLE / FLOW

```
dotnet new classlib -n Fundraising.Domain
```

Open its `.csproj` and identify:

EXAMPLE / FLOW

```
Project SDK
TargetFramework
Nullable
ImplicitUsings
```

Build the project and inspect:

EXAMPLE / FLOW

```
bin/
obj/
```

Identify the generated:

EXAMPLE / FLOW

```
.dll
.pdb
.deps.json
.runtimeconfig.json
```

where applicable to the project type.

Then explain to yourself:

EXAMPLE / FLOW

```
Which file did I write?

Which file defines the project?

What did the compiler generate?

What does CLR do?

What does JIT do?

Why is the DLL called an assembly?
```

If you can explain these without memorizing phrases, you understand the first topic correctly.

Quick Revision

Concept	Meaning
C#	Programming language
.NET	Development/runtime platform
SDK	Tools for developing/building .NET apps
Runtime	Environment needed to execute managed apps
CLR	Managed execution environment

Concept	Meaning
Managed code	Code executing under .NET runtime management
CIL / IL	Intermediate instructions generated from managed source
JIT	Converts IL to native machine code at runtime
Assembly	Compiled `.NET` unit such as DLL/executable
Metadata	Information describing types/members/references
Namespace	Logical grouping of type names
`.csproj`	C# project/build definition
Solution	Container/organization for projects
NuGet	Package management
`dotnet` CLI	Command-line tooling for .NET
Debug	Development/debug-oriented build configuration
Release	Production-oriented optimized build configuration

Complete Final Flow

EXAMPLE / FLOW

```
Developer
    ->
Writes C# files
    ->
Project (.csproj)
    ->
.NET SDK / MSBuild
    ->
C# Compiler
    ->
CIL + Metadata
    ->
Assembly (.dll / executable)
    ->
CLR loads managed application
    ->
JIT converts required CIL to native code
    ->
CPU executes code
    ->
Fundraising application performs business operation
```

This execution model is the foundation underneath almost everything else we will learn in C# and .NET.

The next roadmap topic is: Variables, Data Types and the C# Type System.